

# Cryptographic Architecture Statement: The Agent Witness Protocol (AWP)

## 1. The "Composition, Not Invention" Philosophy

For security architects, the strategic value of a cryptographic protocol is rarely found in the novelty of its primitives. In fact, cryptographic novelty is a liability that introduces unvetted risk. The structural integrity of a system lies instead in the auditability, maturity, and interoperability of established standards. The Agent Witness Protocol (AWP) is predicated on a "composition, not invention" philosophy: weaving together proven cryptographic threads to create a tamper-evident record of agent intent and authorization. By utilizing mature standards, AWP reduces the "attack surface of novelty" and ensures the protocol is resistant to implementation-specific vulnerabilities.

This approach is codified in the 80/20 Rule of AWP, which distinguishes between the commodity infrastructure adopted for reliability and the 20% proprietary innovation that defines AWP's unique value proposition.

| 80% Commodity (Adopted Standards)                                 | 20% Proprietary Innovation (AWP Core)                                      |
|-------------------------------------------------------------------|----------------------------------------------------------------------------|
| DSSE (v1.0): Standardized signing envelope for payload integrity. | WitnessRecord Schema: A structured grammar for governed agent events.      |
| RFC 9162 (SHA256): Standardized Merkle transparency log rules.    | Typed Honesty Boundary: Structural prevention of evidentiary overclaiming. |
| OpenTimestamps (OTS): Trust-minimized Bitcoin temporal anchoring. | Neutral Topology: A non-colluding witness architecture (Producer vs. Log). |
| RFC 3161: eIDAS-aligned temporal proof slots for legal weight.    | Principal Binding: Binding human authorization to specific agent intent.   |

AWP is engineered for re-implementability and zero hidden canonicalization. It avoids the pitfalls of ambiguous serialization by mandating recursive lexicographic

key-sorting and compact JSON separators for all signed payloads. This enables any auditor to re-derive every signed byte from the original specifications, ensuring that the "moat" protecting the protocol is its neutral infrastructure and semantics, not a secret or opaque algorithm. The registration of the [awp.dev](#) domain represents an irreversible wire-format commitment; once published, the protocol transitions from an experimental draft to a stable, interoperable standard.

## 2. The WitnessRecord and the Typed Honesty Boundary

The strategic core of AWP is "Honest Architecture." Traditional logging systems often fail by conflating a "signature" with "truth." AWP solves this by making overclaiming grammatically impossible through its [WitnessRecord](#) schema. By typing the witness's testimony, AWP clarifies the evidentiary weight for auditors and verifiers, ensuring that binary-fail-closed verification is the default posture.

The [claim\\_class](#) taxonomy is the structural enforcement of this honesty, designed specifically to shift and limit reliance liability:

- [integrity-since-witness](#): The baseline claim. It asserts only that the material is unaltered since the moment of witnessing. It attributes truth to no one, effectively stating: "This record hasn't changed."
- [verified-against](#): The witness asserts it checked the material against a specific issuer's trust anchor (e.g., a cryptographic key). Crucially, the liability for the *truth* of the claim remains with the issuer; AWP's liability is strictly limited to the correctness of the cryptographic key check.
- [asserted-by](#): The witness records a claim made by an external issuer without independent verification. The "So What?" for auditors is clear: liability is limited to transcript accuracy—the witness warrants only that it was told this claim, not that the claim is true.

This boundary is enforced in code and verifier output. AWP acts as a pass-through for issuer-asserted assurance levels (`assurance_echo`) and maintains a rigorous set of negative assurances.

AWP is designed to produce evidence, not absolute truth. The protocol and its verifiers **MUST NOT** claim:

- Not Tamper-Proof: Records are *tamper-evident*. AWP detects alteration but does not prevent the physical corruption or deletion of storage.
- Not Authenticity-at-Origin: Proving a record is unaltered *since* witnessing says nothing about whether the data was legitimate or truthful when first encountered.
- Not Identity-Proofing: AWP echoes issuer assurance; it never asserts that a person is "real" or that their identity has been proofed beyond the presented credential.

### 3. Envelope Selection: The Case for DSSE and JSON over COSE/SCITT

A primary architectural decision was the selection of the signing envelope. While industry trends point toward COSE (CBOR Object Signing and Encryption) and the SCITT (Supply Chain Integrity, Transparency, and Trust) profile, AWP remains "SCITT-aligned but not conformant."

AWP utilizes the DSSE (Dead Simple Signing Envelope) and in-toto Statement format. Findings from an eight-agent architectural roundtable concluded that a pivot to COSE\_Sign1 at this stage would produce an "architecturally incoherent chimera." Such a pivot would break existing compatibility with CNCF-graduated in-toto verifiers and add significant development overhead for a moving, pre-RFC target.

The protocol's competitive advantage resides in the payload semantics and the witness's neutral position, not the wire format. To maintain agility, AWP implements a Migration Gate strategy. The decision to export COSE-compatible receipts will only be re-opened if one of the following triggers is met:

1. **Procurement Requirements:** A named enterprise or government partner mandates SCITT/COSE conformance.
2. **RFC Finalization:** The IETF SCITT architecture reaches final RFC status with a stable conformance suite.
3. **Scheduled Review:** A strategic review occurs (no earlier than Q4 2026).

### 4. Hybrid Post-Quantum Cryptography: Additive ML-DSA-44

To ensure long-term architectural durability, AWP adopts an Additive Hybridization posture for Post-Quantum Cryptography (PQC). This approach adds a PQC signature alongside the classical Ed25519 signature, ensuring the protocol is no weaker than classical cryptography alone while providing an experimental PQC backstop.

AWP implements ML-DSA-44 (FIPS-204). Notably, the architecture uses the standardized NIST naming convention rather than research names like "Dilithium2" to serve long-term durability. A critical technical detail is that both the classical and the ML-DSA-44 signatures are calculated over the identical PAE (Pre-Authentication Encoding) bytes. This ensures there is no second canonicalization and no second payload, drastically reducing the implementation surface.

AWP utilizes a **keyid** convention within the DSSE **signatures[ ]** array to enable crypto-agility. This preserves 100% DSSE-v1.0 conformance by avoiding bespoke envelope fields.

None

```
// Keyid convention for crypto-agility
// Backward-compat rule: no prefix (or absent) defaults to
Ed25519
"keyid": "ml-dsa-44:fp" // suite:fingerprint
```

Verification follows a "Classical-Required" rule. During this experimental phase, an Ed25519 signature must always be present and valid. This prevents downgrade attacks where an attacker might strip the classical signature to exploit potentially nascent PQC implementations. The hybrid guarantee ensures an attacker must defeat both algorithms simultaneously to forge a record.

## 5. Neutral Third-Party Witness Topology and Transparency

The AWP architecture recognizes that "Self-Attestation is Documentation, not Evidence." For a record to possess true evidentiary weight—meeting the requirements of the EU AI Act's Article 12—it must be witnessed by a neutral, non-colluding third party.

The "Moat of Neutrality" is established by the protocol's topology, which distinguishes between the Producer's Record Key (used by the agent system) and the Log's Key (used by the neutral steward). Owning the [awp.dev](https://awp.dev) domain and operating as a neutral steward serves as a superior competitive defense compared to a proprietary wire format.

### The Merkle Layer: Standard Raw-Byte Rule

Interoperability is maintained through strict adherence to the RFC9162\_SHA256 standard. AWP employs the "Standard Raw-Byte Rule" for node hashing: `H(0x01 || left || right)`. Architects are cautioned against the non-standard "UTF8-Hex variant" (hashing lowercase hex strings as text) used in some legacy systems; mixing these rules leads to unverifiable roots and catastrophic interoperability failure.

Transparency is further enforced through the C2SP Checkpoint mechanism. Individual records are committed to a Merkle tree, and the root is published in a "signed-note" format. This provides verifiers with authoritative roots for inclusion proofs, linking the [WitnessRecord](#) to a publicly auditable, append-only log.

## 6. Temporal Anchoring: OpenTimestamps and eIDAS Qualification

The final layer of the AWP architecture is the anchor to external time. To ensure scalability, AWP anchors Checkpoint Roots rather than individual records, proving that a batch of events existed at a specific point in time.

| Feature            | OpenTimestamps (OTS)              | RFC 3161 (Qualified eIDAS)             |
|--------------------|-----------------------------------|----------------------------------------|
| Trust Model        | Trust-minimized (Bitcoin)         | Centralized (Qualified Trust Provider) |
| Network Dependency | Online Submit / Offline Verify    | Offline Verify (with pinned anchors)   |
| Verification       | Offline via Bitcoin Block Headers | Offline via Pinned Trust Anchors       |
| Legal Weight       | Functional proof of existence     | Art. 41 Legal Presumption (EU)         |

AWP emphasizes "Honest Asynchrony" in its temporal proofs. For OTS, the verifier explicitly distinguishes between "Calendar-attested" (pending) and "Bitcoin-confirmed" states, ensuring time is never fabricated.

Regarding RFC 3161, the "Qualified" status is an Operator-Pinned assertion. Crucially, the verifier does not infer qualification status from the RFC 3161 token itself. Legal presumption of accuracy is an operator assertion based on out-of-band verification of the Trust Service Provider's status on an EU Trusted List. If a token is verified against an anchor not explicitly configured as "qualified," it is reported only as a plain timestamp.

These layers—Record, Envelope, Log, and Anchor—culminate in a single, offline-verifiable "Receipt" that fulfills the mission of Composition, Not Invention.